

CORSO NODE.JS + EXPRESS + JWT

Fondamenti di Node.js

Dispensa del corso

Giorno 1 — Parte 1

DOCENTE

Massimo Petrolati

Product Manager · Namirial SpA

Edizione 2026 · Documento di studio per studenti livello base

Indice

Cosa troverai in questa dispensa

1	Cos'è Node.js e perché esiste	6
1.1	Una definizione semplice	6
1.2	La storia: chi, quando, perché	6
1.3	Il problema che Node ha risolto	7
1.4	Casi d'uso reali	7
1.5	Quando NON usare Node	8
2	Sotto il cofano: V8 ed Event Loop	10
2.1	Il motore V8	10
2.2	Architettura di Node.js	10
2.3	Single-thread e Event Loop	11
2.4	Un esempio passo passo	11
3	Installazione e primi passi	14
3.1	Installare Node.js	14
3.2	Verificare l'installazione	14
3.3	Il REPL	15
3.4	Il primo script	15
4	Moduli e npm	18
4.1	Cos'è un modulo	18
4.2	CommonJS vs ES Modules	18
4.3	ES Modules in pratica	19
4.4	Le 5 trappole di ESM	20
4.5	I moduli core di Node	21
4.6	npm: il package manager	22
5	Il modulo http: il primo server	25
5.1	Cosa significa "fare un server"	25
5.2	Hello World server	26
5.3	Routing manuale	27
5.4	Parametri GET (query string)	27
5.5	Body POST (stream)	28
5.6	Ponte verso Express	29

Introduzione

Come leggere questa dispensa e cosa porti a casa

Questa dispensa raccoglie tutte le nozioni della **prima parte del Giorno 1** del corso Node.js + Express + JWT. È pensata per essere letta come supporto alle slide della lezione, oppure come materiale di studio autonomo dopo l'aula.

Il livello di partenza richiesto è **base**: serve solo aver scritto un po' di JavaScript, conoscere variabili, funzioni, array, oggetti e qualche nozione di asincronia (callback o promise). Tutto il resto viene spiegato a partire dal primo principio.

Come è strutturata

La dispensa segue gli stessi 5 blocchi delle slide: cos'è Node, come funziona dentro, come si installa, come si gestiscono i moduli e infine come si costruisce un primo server HTTP. Ogni sezione contiene una spiegazione discorsiva, esempi di codice commentati e box di approfondimento.

I box che incontrerai

- **CODICE**: riferimento al mini-progetto pronto nella cartella `codice-esempi/giorno-1/`. Apri il file, eseguillo, modificalo.
- **Suggerimento**: consiglio pratico per evitare problemi tipici.
- **Attenzione**: trappola comune o cosa NON fare.

Come imparare davvero

Leggere la teoria serve a poco se poi non si scrive codice. L'unico modo per fissare i concetti è **aprire un terminale, scrivere gli esempi a mano**, vederli funzionare e provare a modificarli. Se qualcosa non torna, non passare avanti: fermati e capisci perché. È normale che all'inizio i concetti sembrino tutti collegati e confusi: diventano chiari solo dopo qualche ora di pratica.

Dove trovi il codice

Tutti gli esempi della dispensa sono pronti nel **repository GitHub del corso**:

github.com/mpetrolati/2026-corso-backend-nodejs-express-its

Clonalo **una volta sola** all'inizio del corso e usalo come riferimento per ogni esempio:

```
$ git clone https://github.com/mpetrolati/2026-corso-backend-nodejs-express-its.git  
$ cd 2026-corso-backend-nodejs-express-its
```

I file di questa dispensa stanno nella cartella `codice-esempi/giorno-1/`.

CAPITOLO 01

Cos'è Node.js e perché esiste

Storia, casi d'uso e limiti del runtime

1. Cos'è Node.js e perché esiste

1.1 Una definizione semplice

Node.js è un runtime JavaScript lato server, basato sul motore V8 di Google Chrome.

Sembra una definizione piena di parole tecniche, ma ognuna ha un significato preciso:

- **Runtime:** un ambiente di esecuzione, cioè qualcosa che permette di lanciare codice. Pensa a Node come al “motore” che fa girare i tuoi file JavaScript fuori dal browser.
- **JavaScript:** il linguaggio che già conosci (almeno un po'). Le variabili, le funzioni, gli `if`, i `for` sono identici a quelli che useresti in un file `.js` dentro una pagina HTML.
- **Lato server:** il codice non gira nel browser dell'utente, ma su un computer remoto (o sul tuo, durante lo sviluppo). Risponde a richieste, gestisce database, processa file.
- **Basato su V8:** dentro a Node c'è il motore JavaScript di Chrome. Quando scrivi `node app.js`, è V8 che compila ed esegue il tuo codice.

La cosa più importante da capire: **Node non è un linguaggio nuovo e non è un framework.** È solo un modo per eseguire JavaScript fuori dal browser. La sintassi che useremo è la stessa del JS che gira in Chrome, con qualche differenza nelle API disponibili (in Node non esistono `window` o `document`, ma esistono `process`, `fs`, ecc.).

1.2 La storia: chi, quando, perché

Node.js è stato creato da **Ryan Dahl** e presentato alla **JSConf EU del 2009**. All'epoca, JavaScript era considerato un linguaggio “da browser”, spesso snobbato per fare software serio. Ryan ebbe un'intuizione potente: prendere il motore V8 (che era veloce, perché serviva a Chrome per fare girare Gmail in tempo reale) e usarlo come base per scrivere server di rete.

Il punto di forza scelto era una **API non bloccante basata sugli eventi**: mentre i server tradizionali (Apache, ad esempio) usavano un thread per ogni connessione, Node proponeva di usare un solo thread che, invece di stare fermo ad aspettare i risultati delle operazioni lente, si limitava a “ordinarle” al sistema operativo e a essere notificato quando erano pronte. Risultato: stesso hardware, molte più connessioni concorrenti gestibili.

Negli anni successivi Node è esploso. Oggi è usato da Netflix, LinkedIn, PayPal, Uber, NASA e da praticamente ogni startup. È il runtime dietro a npm (il più grande registro di pacchetti software del mondo) e dietro a tool che probabilmente già usi senza saperlo: **Webpack**, **Vite**, **Babel**, **ESLint**, **Prettier** girano tutti su Node.

1.3 Il problema che Node ha risolto

Per capire perché Node è nato, immagina di gestire un ristorante. Hai 100 tavoli da servire, ma un'unica regola: ogni cameriere segue un solo tavolo dall'inizio alla fine. Se il cliente ordina un piatto che richiede 30 minuti di cottura, il cameriere rimane fermo accanto al tavolo per 30 minuti, aspettando. Se hai 100 tavoli, ti servono 100 camerieri. Costano tanto e per la maggior parte del tempo non fanno niente.

Questo è il modello dei **server tradizionali** stile Apache: un thread (=un cameriere) per ogni richiesta (=un tavolo). Quando la richiesta è lenta perché aspetta il database o un file da disco, il thread rimane bloccato. Con migliaia di utenti, il numero di thread esplode e la memoria si satura.

Node propone un modello diverso: **un solo cameriere super-veloce**. Quando un cliente ordina qualcosa di lento, il cameriere prende la nota, la consegna in cucina e passa subito al tavolo successivo. Quando la cucina è pronta, suona un campanello e il cameriere torna a servire quel tavolo. Un solo cameriere riesce a gestire 100 tavoli senza problemi, perché non sta mai fermo ad aspettare.

In termini tecnici

Node usa un **singolo thread** per eseguire il tuo codice JavaScript, e delega le operazioni di I/O (input/output: lettura file, query DB, richieste di rete) al sistema operativo o a un pool di thread interno. Quando l'I/O è pronto, Node viene notificato e richiama la tua funzione (la *callback*). Questo modello si chiama **event-driven, non-blocking I/O**.

1.4 Casi d'uso reali

Node.js brilla in tutti gli scenari dove la richiesta principale è gestire **tante connessioni concorrenti che fanno per lo più I/O** (cioè leggono/scrivono dati, non fanno calcoli pesanti sulla CPU):

- **API REST**: il caso d'uso più comune. Server che ricevono richieste HTTP, leggono o scrivono nel database e rispondono con JSON. È quello che faremo in questo corso.
- **Real-time / chat / collaborazione**: applicazioni dove utenti diversi interagiscono in tempo reale. WhatsApp Web, Slack, strumenti di disegno collaborativo usano Node sotto.
- **Microservizi**: tanti piccoli servizi che parlano tra loro via HTTP o messaggi. Node permette di scriverli velocemente e di farli partire con pochi MB di memoria.
- **Tool di sviluppo**: praticamente tutti i build tool e i linter moderni girano su Node, perché lavorano su file (I/O) e devono essere veloci da lanciare.
- **Streaming di dati**: Node ha un modello nativo per gestire flussi di dati grandi (stream). Netflix, ad esempio, lo usa per servire video.

1.5 Quando NON usare Node

Node è ottimizzato per fare poche operazioni CPU su tante connessioni. **Si rompe in scenari opposti:** tante operazioni CPU su poche connessioni. Il single-thread, infatti, diventa un collo di bottiglia: se il tuo codice JavaScript impiega 30 secondi a finire un calcolo, durante quei 30 secondi **nessun'altra richiesta viene servita**.

Tipici casi in cui Node non è la scelta giusta:

- **Elaborazione video/immagini pesante:** encoding, transcoding, rendering. Meglio usare strumenti dedicati (ffmpeg, GPU). Se proprio devi farlo da Node, delega il lavoro a un processo esterno o a un worker thread.
- **Machine learning, training di modelli:** i calcoli matriciali sono enormi. L'ecosistema Python (NumPy, TensorFlow, PyTorch) è enormemente più maturo.
- **Calcoli scientifici intensivi:** simulazioni numeriche, algoritmi pesanti. Linguaggi compilati (C++, Rust, Go) sono molto più veloci.
- **Operazioni CPU-bound nel ciclo della richiesta:** se per rispondere a un utente devi fare un calcolo che blocca la CPU per più di qualche decina di millisecondi, stai bloccando l'event loop e tutti gli altri client si fermano.

Regola pratica

Se ti chiedi se Node è adatto al tuo caso, fatti questa domanda: **quanto tempo passa il mio codice JavaScript a fare calcoli, rispetto al tempo che passa ad aspettare risultati da database, rete o file?** Se la risposta è “quasi tutto in attesa”, Node è perfetto. Se la risposta è “quasi tutto a calcolare”, probabilmente c'è di meglio.

CAPITOLO 02

Sotto il cofano: V8 ed Event Loop

Come funziona davvero Node, spiegato con calma

2. Sotto il cofano: V8 ed Event Loop

2.1 Il motore V8

V8 è il motore JavaScript open-source sviluppato da Google per il browser Chrome. È scritto in C++ ed è il responsabile di prendere il tuo codice JavaScript e farlo eseguire dal processore del computer.

Storicamente, JavaScript era un linguaggio **interpretato**: il browser leggeva il codice una riga alla volta e lo eseguiva. Funzionava, ma era lento. V8 ha introdotto la **compilazione JIT (Just-In-Time)**: il codice viene compilato in codice macchina al volo, e le parti eseguite più spesso vengono ottimizzate progressivamente. Risultato: JavaScript è diventato sorprendentemente veloce, paragonabile a linguaggi compilati per molte operazioni.

Curiosità

Non solo Chrome e Node usano V8: anche Microsoft Edge (la versione moderna basata su Chromium), Brave, Opera e tantissime altre app desktop costruite con Electron (VS Code, Slack, Discord, WhatsApp Desktop) usano V8 sotto. Tu impari un linguaggio e lo usi ovunque.

Per essere precisi, esistono anche altri motori JavaScript validi: **SpiderMonkey** (Firefox), **JavaScriptCore** (Safari), **QuickJS**. Esistono anche runtime alternativi a Node che usano motori diversi: **Deno** e **Bun** sono i più conosciuti. Ma nel corso useremo sempre Node + V8, perché è lo standard di mercato.

2.2 Architettura di Node.js

Node non è solo V8. È un pacchetto che mette insieme V8 con altre librerie e API. Possiamo immaginarlo come un edificio a 4 piani:

- **Il tuo codice JavaScript** (il piano più alto): `app.js`, route, controller, business logic. È quello che scriverai tu.
- **API di Node.js** (terzo piano): moduli built-in come `http`, `fs`, `path`, `os`. Sono scritti in JavaScript e fanno da ponte verso il sistema operativo.
- **Bindings** (secondo piano): codice C++ che collega il mondo JS alle librerie native.
- **V8 + libuv** (piano terra): V8 esegue il JavaScript. **libuv** è una libreria C che fornisce l'**event loop** e un **pool di thread** per gestire le operazioni di I/O bloccanti (file system, DNS, alcune operazioni di crittografia).

La parte interessante è **libuv**: quando dici a Node "leggi questo file", Node non sta lì a leggerlo riga per riga. Delega il lavoro a libuv, che lo passa al sistema operativo o a un thread del pool.

Quando il file è pronto, libuv mette la callback nella coda dell'event loop, che la eseguirà appena V8 è libero.

2.3 Single-thread e Event Loop

Il concetto chiave da fissare bene: **il tuo codice JavaScript in Node gira su un singolo thread**. Non c'è il rischio che due funzioni vengano eseguite contemporaneamente. Non hai bisogno di mutex, lock o race condition come in altri linguaggi. Questo è un grande vantaggio in termini di semplicità.

Ma allora come fa Node a gestire migliaia di richieste simultanee? La risposta è **asincronia**. Quando il tuo codice chiede un'operazione di I/O (esempio: leggere un file da disco), Node non aspetta il risultato bloccando tutto. Invece:

- Node passa la richiesta a libuv (che la affida al sistema operativo o al thread pool).
- Il tuo thread JavaScript è libero di continuare con altro codice.
- Quando l'operazione è completata, libuv mette la **callback** nella **coda dell'event loop**.
- Appena il tuo codice sincrono finisce, l'event loop preleva la callback dalla coda e la esegue.

Pensa così

Immagina di essere a fare il bucato in lavanderia. Versione bloccante: stai 40 minuti fermo a fissare la lavatrice. Versione asincrona (Node): metti il bucato, fai partire la lavatrice, vai a fare la spesa, e quando la lavatrice suona torni a stenderlo. Stessa lavatrice, stesso bucato, ma in 40 minuti hai fatto anche la spesa.

Esistono diverse "code" all'interno dell'event loop, ognuna con priorità diversa. Le più importanti da conoscere a livello base sono:

- **Codice sincrono**: viene eseguito sempre prima di qualsiasi cosa asincrona.
- **Microtask**: callback di Promise (`.then()`) e `process.nextTick()`. Vengono eseguite subito dopo il sincrono, prima di tornare al loop principale.
- **Macrotask**: callback di `setTimeout`, `setImmediate`, callback di I/O. Vengono processate nel prossimo "giro" dell'event loop.

2.4 Un esempio passo passo

Vediamo concretamente come Node decide l'ordine di esecuzione. Considera questo codice:

```
console.log('1');
setTimeout(() => console.log('2'), 0);
setImmediate(() => console.log('3'));
process.nextTick(() => console.log('4'));
Promise.resolve().then(() => console.log('5'));
console.log('6');
```

Quale sarà l'output? La tentazione è di rispondere “1, 2, 3, 4, 5, 6”, leggendolo dall’alto in basso. Ma l'output reale è: **1, 6, 4, 5, 2, 3**.

Vediamo perché, riga per riga:

- **1** esce per primo: è codice sincrono, semplice `console.log`.
- **setTimeout(... 0)** NON esce: la callback viene messa in coda nella categoria “macrotask”.
- **setImmediate(...)** NON esce: stessa cosa, va in coda macrotask.
- **process.nextTick(...)** NON esce: la callback va in coda microtask, con priorità massima.
- **Promise.resolve().then(...)** NON esce: callback in coda microtask.
- **6** esce subito dopo 1: è codice sincrono.
- **4** esce dopo 6: il sincrono è finito, ora si processano le microtask, e `nextTick` ha priorità più alta di Promise.
- **5** esce dopo 4: è l'altra microtask in coda.
- **2** esce dopo 5: terminate le microtask, si passa alle macrotask. `setTimeout` viene di solito processato prima di `setImmediate`.
- **3** esce per ultimo: macrotask di `setImmediate`.

La regola da memorizzare

Prima tutto il **codice sincrono**, poi tutte le **microtask** (`nextTick`, `Promise`), infine le **macrotask** (`setTimeout`, `setImmediate`). All'inizio sembrerà controintuitivo, ma una volta che hai questa regola in testa, tutto torna.

CODICE: 02-event-loop/event-loop-demo.js

CAPITOLO 03

Installazione e primi passi

Setup del runtime, REPL e primo script

3. Installazione e primi passi

3.1 Installare Node.js

Il modo ufficiale per installare Node è scaricare l'installer dal sito ufficiale:

```
https://nodejs.org
```

All'apertura del sito troverai due versioni proposte:

- **LTS (Long Term Support):** la versione stabile, supportata per anni. Al momento attuale è la 20.x o 22.x. **È quella che useremo nel corso e che ti consiglio per qualsiasi progetto serio.**
- **Current:** l'ultima versione disponibile, con le feature più recenti ma meno stabile. Sconsigliata in produzione, può andare bene per sperimentare.

L'installer è disponibile per tutti i sistemi operativi principali:

- **Windows:** file `.msi`. Durante l'installazione lascia spuntata l'opzione "Add to PATH": serve per poter usare il comando `node` da qualsiasi terminale.
- **macOS:** file `.pkg`. Doppio click e via, non servono configurazioni.
- **Linux:** puoi scaricare i binari pronti o usare il package manager della tua distro (apt, yum, dnf).

Suggerimento

Insieme a Node viene installato anche **npm** (il package manager). Non devi installarlo a parte. Se invece in futuro vorrai gestire più versioni di Node contemporaneamente, potrai valutare strumenti come **nvm** (macOS/Linux) o **nvm-windows**. Per ora non serve.

Attenzione

Dopo aver installato Node, **devi aprire un NUOVO terminale**. Le variabili d'ambiente del sistema si aggiornano solo all'apertura di una nuova shell. Se provi a usare `node` nella finestra che era già aperta prima dell'installazione, non lo troverà.

3.2 Verificare l'installazione

Apri un terminale (Terminal su macOS/Linux, PowerShell o cmd su Windows) e scrivi:

```
$ node -v
v22.0.0

$ npm -v
10.5.0
```

Se vedi una versione, sei a posto. Se ricevi un errore tipo “comando non trovato”, controlla:

- Hai aperto un terminale NUOVO dopo l'installazione?
- Su Windows: controlla che `C:\Program Files\nodejs\` sia nel PATH.
- Su macOS/Linux: di solito Node finisce in `/usr/local/bin/`, che è già nel PATH.

3.3 Il REPL

Se lanci `node` da solo, senza argomenti, entri in un **REPL (Read-Eval-Print Loop)**: un ambiente interattivo dove puoi scrivere JavaScript e vederne il risultato immediatamente. È utile per prove veloci.

```
$ node
> 2 + 2
4
> Math.sqrt(144)
12
> const x = 10
undefined
> x * 5
50
> .exit
```

Per uscire dal REPL: `.exit` oppure **Ctrl+C** due volte.

3.4 Il primo script

Per scrivere uno script vero e proprio, crea un file `hello.js` con questo contenuto:

```
console.log('Hello Node!');
console.log('File:', __filename);
console.log('Versione di Node:', process.version);
```

Salva ed esegui dal terminale:

```
$ node hello.js
```

Vedrai i tre messaggi stampati. Hai appena scritto ed eseguito il tuo primo script Node. Da qui in poi, ogni volta che diciamo “esegui questo file”, intendiamo lanciare `node nome-del-file.js` dal terminale.

Cosa cambia rispetto al browser

In Node NON esistono `window`, `document`, `alert`. Esistono però altre variabili globali utili: `process` (info sul processo), `global` (l'equivalente di `window`), e `__dirname` (path della cartella e del file corrente).

CODICE: 01-hello/hello.js

CAPITOLO 04

Moduli e npm

Spezzare il codice e usare codice altrui

4. Moduli e npm

4.1 Cos'è un modulo

Un **modulo**, in Node, è semplicemente un file JavaScript che esporta qualcosa (funzioni, variabili, classi) e che può essere importato da altri file. Serve a **spezzare un programma in pezzi più piccoli e riutilizzabili**.

Senza moduli, dovresti scrivere tutto il tuo programma in un solo file gigantesco. Con i moduli puoi avere `database.js`, `auth.js`, `utils.js`, ecc., ognuno con una sua responsabilità precisa. Il vantaggio cresce esponenzialmente con la dimensione del progetto.

4.2 CommonJS vs ES Modules

In Node convivono **due sistemi diversi di moduli**. Li elencheremo brevemente, perché li incontrerai entrambi guardando codice online, ma ne useremo solo uno nel corso.

CommonJS (CJS) — il sistema storico

È il sistema con cui Node è nato. Usa `require()` per importare e `module.exports` per esportare.

```
// math.js
function somma(a, b) { return a + b; }
module.exports = { somma };

// app.js
const { somma } = require('./math');
console.log(somma(2, 3));
```

ES Modules (ESM) — lo standard moderno

È la sintassi che JavaScript ha adottato come standard ufficiale. Usa `import` e `export`. È più moderna, più potente, e **la useremo per tutto il corso**.

```
// math.js
export function somma(a, b) { return a + b; }

// app.js
import { somma } from './math.js';
console.log(somma(2, 3));
```

Perché scegliere ESM:

- È lo **standard JavaScript**: oggi gira sia nei browser che in Node, senza differenze.
- Sintassi più pulita e leggibile.
- Supporta funzionalità moderne come **top-level await** (puoi scrivere `await` direttamente al livello principale del file).
- Nelle nuove librerie e tutorial troverai sempre più solo ESM.

4.3 ES Modules in pratica

Per dire a Node “questo progetto usa ES Modules”, basta una riga nel `package.json` del progetto:

```
{
  "name": "mio-progetto",
  "type": "module"
}
```

Con questa riga, tutti i file `.js` dentro il progetto vengono interpretati come ES Modules. Senza, Node li tratterebbe come CommonJS.

Tipi di export

ES Modules supporta due modi di esportare:

- **Named export**: esporti più cose, ognuna con un nome esplicito. Si importano con le graffe.
- **Default export**: esporti un'unica cosa di default. Si importa senza graffe, puoi darle il nome che vuoi.

```
// math.js – entrambi insieme
export function somma(a, b) { return a + b; }
export const PI = 3.14159;
export default function moltiplica(a, b) { return a * b; }

// app.js – diversi modi di importare
import { somma, PI } from './math.js';           // named
import moltiplica from './math.js';              // default
import * as math from './math.js';               // tutto in un oggetto
import miaFunzione from './math.js';             // default rinominato
```

CODICE: 03-modules/ (math.js + app.js + package.json)

4.4 Le 5 trappole di ESM

ES Modules è più moderno di CommonJS, ma ha alcune particolarità che possono far perdere tempo a chi non le conosce. Vediamone cinque da tenere a mente.

1. "type": "module" è obbligatorio

Senza la riga `"type": "module"` nel `package.json`, Node interpreta i file `.js` come CommonJS e ti dà errori del tipo “Cannot use import statement outside a module”.

2. L'estensione .js è obbligatoria negli import relativi

In ESM devi sempre scrivere l'estensione del file negli import relativi:

```
import { somma } from './math.js'; // funziona
import { somma } from './math';    // ERR_MODULE_NOT_FOUND
```

Questa è la trappola più frequente per chi viene da React, Vue o altri framework dove i bundler (Webpack, Vite) permettono di omettere l'estensione. Node è più rigoroso.

3. __dirname e __filename non esistono

In CommonJS hai a disposizione di default `__dirname` e `__filename`. In ESM non ci sono. Per ricostruirli serve questo piccolo boilerplate:

```
import { fileURLToPath } from 'url';
import { dirname } from 'path';

const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);
```

Tienilo nei tuoi snippet, ti servirà ogni volta che dovrai servire file statici o caricare template a partire dal path del file corrente.

4. Import di JSON richiede sintassi speciale

Importare un file JSON non funziona come ci si aspetta. Da Node 22 in poi la sintassi corretta è:

```
import data from './data.json' with { type: 'json' };
```

In alternativa, puoi sempre usare `fs.readFileSync` + `JSON.parse`. Su versioni più vecchie di Node era `assert { type: 'json' }`.

5. Top-level await è disponibile

Questa è una bella notizia: in ESM puoi usare `await` direttamente al livello principale del file, senza dover wrappare tutto in una funzione `async`. Comodo per operazioni di setup all'avvio dell'app, tipo connettersi al database.

```
const data = await fetch('https://api.example.com/data').then(r => r.json());
console.log(data);
```

4.5 I moduli core di Node

Node viene con una libreria standard ricca di moduli pronti all'uso. Si chiamano “core modules” e non hanno bisogno di essere installati: basta importarli. Eccone una carrellata di quelli che useremo nel corso o che vedrai spesso in giro.

fs — File System

Per leggere e scrivere file. Ne esistono due versioni: `fs` con callback (vecchia scuola) e `fs/promises` con `async/await` (moderna). **Usa sempre la versione `promises`.**

```
import { readFile, writeFile } from 'fs/promises';

await writeFile('output.txt', 'Ciao mondo!', 'utf-8');
const contenuto = await readFile('output.txt', 'utf-8');
console.log(contenuto);
```

path — Path cross-platform

Per costruire percorsi di file che funzionano sia su Windows (separatore “\”) che su macOS/Linux (“/”).

```
import path from 'path';

const p = path.join('cartella', 'sotto', 'file.txt');
console.log(p);
// Su macOS/Linux: cartella/sotto/file.txt
// Su Windows: cartella\sotto\file.txt
```

os — Informazioni sul sistema

Per ottenere info sul sistema operativo, CPU, memoria, utente.

```
import os from 'os';

console.log('Piattaforma:', os.platform());
console.log('CPU:', os.cpus().length, 'core');
console.log('RAM totale:', (os.totalmem() / 1024 ** 3).toFixed(2), 'GB');
```

events — EventEmitter

Il pattern degli eventi (publish/subscribe) è al cuore di Node. Tantissimi oggetti, incluso `http.Server` e `req`, ereditano da `EventEmitter`. Capirlo qui significa capire come funziona tutta l'API I/O di Node.

url — Analisi URL

Per “smontare” un URL nei suoi componenti (path, query string, host). Vedremo questo modulo nel capitolo 5 quando leggeremo i parametri GET.

http — Il server HTTP

Il modulo che ci permetterà di creare il nostro primo web server. Tutto il capitolo 5 ci ruota attorno.

CODICE: 04-core-modules/ (fs, path, os, events)

4.6 npm: il package manager

npm sta per *Node Package Manager*. È due cose insieme:

- Un **registro pubblico online** (npmjs.com) dove sviluppatori di tutto il mondo pubblicano librerie open source.
- Una **CLI** (il comando `npm`) che ti permette di installare quelle librerie nel tuo progetto e di gestirne le dipendenze.

È il più grande registro di pacchetti software al mondo: oltre 3 milioni di librerie disponibili. Praticamente per qualsiasi cosa tu voglia fare, esiste già una libreria.

I comandi essenziali

```
# Crea un nuovo progetto (genera package.json con valori di default)
$ npm init -y

# Installa una libreria e la salva in dependencies
$ npm install dayjs

# Installa una libreria solo per lo sviluppo (in devDependencies)
$ npm install --save-dev nodemon

# Installa tutte le dipendenze definite nel package.json
# (utile quando cloni un progetto da GitHub)
$ npm install

# Esegui uno script definito in package.json
$ npm start
$ npm run lint # per script con nome diverso da start/test
```

File e cartelle che troverai

- **package.json**: il manifest del progetto. Contiene nome, versione, autore, dipendenze, script. È un file di testo che committi su Git.
- **package-lock.json**: contiene le versioni esatte di tutte le dipendenze (anche quelle transitive). Garantisce che tu e i tuoi colleghi abbiate esattamente le stesse versioni installate. **VA committato su Git**.
- **node_modules/**: la cartella dove npm scarica fisicamente il codice delle dipendenze. Può diventare enorme (centinaia di MB). **NON va committata su Git**: mettila sempre nel `.gitignore`.

Suggerimento

Esistono alternative a npm con vantaggi diversi: **yarn** (più veloce, gestione workspace), **pnpm** (efficiente in termini di disco, condivide pacchetti tra progetti). Per il corso resteremo su npm perché è quello che trovi di default con Node e perché tutti i comandi sono identici.

CODICE: 05-npm-intro/ (esempio con dayjs)

CAPITOLO 05

Il modulo http: il primo server

Routing, parametri GET, body POST e la strada verso Express

5. Il modulo http: il primo server

5.1 Cosa significa "fare un server"

Un **server** è un programma che resta sempre in ascolto su una porta del computer (per esempio la 3000) e che **risponde a richieste** in arrivo dai client. Il protocollo standard del web si chiama **HTTP**.

Anatomia di una richiesta HTTP

Quando il tuo browser visita `https://google.com`, sotto al cofano manda una **richiesta HTTP** al server di Google. Una richiesta contiene:

- **Metodo:** GET (lettura), POST (creazione), PUT (sostituzione), PATCH (modifica parziale), DELETE (cancellazione). Sono i verbi che esprimono cosa vuoi fare.
- **URL:** l'indirizzo della risorsa, fatto di path (es. `/api/users`) ed eventuali parametri di query (es. `?nome=Mario`).
- **Headers:** metadati della richiesta. Authorization (token), Content-Type (formato del body), Accept-Language, ecc.
- **Body:** il contenuto, presente solo nei metodi POST/PUT/PATCH. Tipicamente JSON nelle API moderne.

Anatomia di una risposta HTTP

Il server risponde con:

- **Status code:** un numero a 3 cifre che indica l'esito. **2xx** = successo (200 OK, 201 Created, 204 No Content). **4xx** = errore del client (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found). **5xx** = errore del server (500 Internal Server Error).
- **Headers:** ancora metadati. Content-Type, Cache-Control, Set-Cookie.
- **Body:** il contenuto della risposta. HTML, JSON, immagini, qualsiasi cosa.

Un punto da fissare

Il **browser** è solo uno dei tanti client HTTP possibili. Altri client sono **curl** (CLI), **Postman** (GUI), **fetch()** da un'altra applicazione, un'app mobile. Quando costruiamo un'API server, non importa chi farà le richieste: il contratto è sempre lo stesso (HTTP).

5.2 Hello World server

Vediamo concretamente come si scrive il server più piccolo possibile in Node, senza alcun framework. Solo Node nudo e crudo:

```
import http from 'http';

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello world!');
});

server.listen(3000, () => {
  console.log('Server in ascolto su http://localhost:3000');
});
```

Cosa succede riga per riga:

- **http.createServer(callback)**: crea un nuovo oggetto server. La callback viene chiamata per ogni richiesta in arrivo e riceve due parametri: `req` (la richiesta) e `res` (la risposta da costruire).
- **res.statusCode = 200**: imposta il codice di risposta. 200 = OK.
- **res.setHeader('Content-Type', 'text/plain')**: dice al client che stiamo mandando testo semplice.
- **res.end('Hello world!')**: scrive il body e chiude la risposta. Senza chiamare `end()`, il client resta lì ad aspettare e la connessione non si chiude mai.
- **server.listen(3000, callback)**: mette il server in ascolto sulla porta 3000. La callback viene eseguita quando il server è pronto.

Lancia con `node server.js` e apri il browser su `http://localhost:3000`. Dovresti vedere “Hello world!”.

Suggerimento

Per fermare il server: **Ctrl+C** nel terminale. Se ricevi l'errore *EADDRINUSE: port 3000 already in use*, significa che hai già un'altra istanza in esecuzione (magari da una finestra precedente). Chiudila prima di rilanciare.

CODICE: 06-http-hello/server.js

5.3 Routing manuale

Un server vero ha più di un endpoint. Per esempio, una home a `/` e una pagina about a `/about`. Con il modulo `http` puro, dobbiamo distinguere a mano le route con catene di `if/else`.

```
const server = http.createServer((req, res) => {
  if (req.url === '/' && req.method === 'GET') {
    res.end('Home');
  } else if (req.url === '/about' && req.method === 'GET') {
    res.end('About');
  } else {
    res.statusCode = 404;
    res.end('Not Found');
  }
});
```

Funziona, ma è subito chiaro che **non scala**. Immagina di avere 20 endpoint: il file diventa un mostro illeggibile. Inoltre dobbiamo gestire a mano:

- Lo status code corretto per ogni caso (200, 404, ecc.).
- Gli header (Content-Type) ogni volta.
- Il parsing dei parametri di URL.
- Il parsing del body se serve.
- Gli errori imprevisti.

Questa è esattamente la frustrazione che ha portato alla nascita di **Express**: un piccolo framework che gestisce tutto questo lavoro ripetitivo per noi. Lo introdurremo nella seconda parte della giornata.

CODICE: 07-http-routing/server.js

5.4 Parametri GET (query string)

Quando un client manda una richiesta con parametri nella URL, tipo:

```
GET /saluta?nome=Mario&cognome=Rossi
```

La parte dopo il `?` è la **query string**. Per leggerla in Node usiamo l'oggetto globale `URL`, che ci permette di “smontare” la richiesta nei suoi componenti.

```
const server = http.createServer((req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);

  if (url.pathname === '/saluta') {
    const nome = url.searchParams.get('nome') || 'sconosciuto';
    const cognome = url.searchParams.get('cognome') || '';
    res.end(`Ciao ${nome} ${cognome}!`);
  }
});
```

Cosa fanno le parti chiave:

- **new URL(req.url, base):** crea un oggetto URL completo. Il secondo argomento è la base, necessaria perché `req.url` è solo il path.
- **url.pathname:** la parte del percorso, senza i parametri (es. `/saluta`).
- **url.searchParams.get('nome'):** estrae il valore del parametro `nome`. Se non c'è, restituisce `null`.

Suggerimento

I parametri di query sono sempre **stringhe**, anche se hanno un aspetto numerico. Se ti aspetti un numero (es. `?eta=30`) ricordati di convertirlo con `Number(valore)` o `parseInt(valore, 10)`.

CODICE: 08-http-get-params/server.js

5.5 Body POST (stream)

Le richieste POST trasportano un body. Per esempio, quando registri un utente, il client manda:

```
POST /api/auth/register
Content-Type: application/json

{ "email": "mario@example.com", "password": "..."}
}
```

Leggere il body in Node “puro” è più complicato di quanto sembri, perché **il body NON arriva tutto insieme**: arriva a pezzi (chunk) via stream. Dobbiamo registrarci su due eventi: uno per ricevere i chunk man mano che arrivano, e uno per sapere quando il body è completo.

```
if (req.method === 'POST' && req.url === '/echo') {  
  let body = '';  
  
  req.on('data', (chunk) => {  
    body += chunk.toString();  
  });  
  
  req.on('end', () => {  
    try {  
      const dati = JSON.parse(body);  
      res.end(JSON.stringify({ ricevuto: dati }));  
    } catch (err) {  
      res.statusCode = 400;  
      res.end(JSON.stringify({ errore: 'JSON non valido' }));  
    }  
  });  
}
```

Cosa accade:

- **req.on('data', ...):** viene chiamato ogni volta che arriva un pezzo di body. Accumuliamo i chunk in una variabile.
- **req.on('end', ...):** viene chiamato quando il body è completo. Solo ora possiamo fare `JSON.parse`.
- **try/catch:** il parsing può fallire se il client ha mandato JSON malformato. Rispondiamo con 400 Bad Request.

Perché Express, in tre righe

Hai notato quante righe ci sono volute solo per leggere un body JSON? Immagina di farlo su 20 endpoint. In Express questo si scrive in 3 righe: `app.use(express.json())` + `app.post('/echo', (req, res) => res.json(req.body))`. Ecco perché useremo Express dalla prossima parte del corso.

CODICE: 09-http-post-body/server.js

5.6 Ponte verso Express

Abbiamo visto che il modulo `http` nativo permette di costruire un server, ma con due limiti che si fanno sentire subito:

- Il **routing** va gestito a mano con if/else, e diventa ingestibile dopo poche route.
- Il **parsing del body** è verboso e ripetitivo.
- **Mancano helper** per gestire JSON, file statici, errori, autenticazione.

Express è un piccolissimo framework (poche centinaia di KB) che si mette sopra al modulo `http` e introduce un'API molto più ergonomica. Lo stesso server che abbiamo visto in queste pagine, in Express si scrive così:

```
import express from 'express';

const app = express();
app.use(express.json());

app.get('/', (req, res) => res.send('Home'));
app.get('/about', (req, res) => res.send('About'));
app.get('/saluta', (req, res) => {
  const nome = req.query.nome || 'sconosciuto';
  res.send(`Ciao ${nome}!`);
});
app.post('/echo', (req, res) => res.json({ ricevuto: req.body }));

app.listen(3000, () => console.log('Server su http://localhost:3000'));
```

Confronta queste righe con quanto codice ci era voluto in Node puro. È questo il motivo per cui praticamente **tutte le API REST scritte in Node usano Express** o un suo concorrente (Fastify, Koa, NestJS, ma il pattern è simile).

Prossima tappa

Nella seconda parte del Giorno 1 installeremo Express, riscriveremo il server che abbiamo appena visto, e introdurremo concetti come **middleware**, **router modulari** e **gestione centralizzata degli errori**. È lì che inizia il lavoro “da backend developer” vero e proprio.

Appendice

Risorse e prossimi passi

Cosa ti porti a casa da questa lezione

- Node.js è un **runtime JavaScript lato server** basato su V8.
- Il suo modello è **single-thread + I/O asincrono**: questo lo rende ottimo per API e applicazioni real-time, ma sbagliato per task CPU-intensive.
- L'**Event Loop** decide l'ordine di esecuzione: prima sincrono, poi microtask, infine macrotask.
- Usiamo **ES Modules** come standard. Ricorda: `"type": "module"` nel `package.json` e `.js` obbligatorio negli import.
- **npm** gestisce le dipendenze. `node_modules` non si committa, `package-lock.json` sì.
- Il modulo **http** permette di fare server senza framework, ma è verboso. Per questo arriva Express.

Cartella codice di riferimento

Tutti gli esempi sono pronti nella cartella `codice-esempi/giorno-1/` del repository GitHub del corso:

github.com/mpetrolati/2026-corso-backend-nodejs-express-its

Se non l'hai ancora fatto, clonalo (o scaricalo come ZIP da GitHub):

```
$ git clone https://github.com/mpetrolati/2026-corso-backend-nodejs-express-its.git
$ cd 2026-corso-backend-nodejs-express-its/codice-esempi/giorno-1
```

Ogni sottocartella è un mini-progetto autonomo con un suo `README.md` e un suo `package.json`. Aprili, lanciali, modificali. È il modo migliore per consolidare quanto hai letto.

- `01-hello/` il primo script
- `02-event-loop/` esempio passo passo dell'event loop
- `03-modules/` ES Modules con export e import
- `04-core-modules/` fs, path, os, events
- `05-npm-intro/` primo progetto con dipendenza esterna

- `06-http-hello/` Hello World server
- `07-http-routing/` routing manuale
- `08-http-get-params/` parametri GET
- `09-http-post-body/` body POST con stream

Per approfondire

- **Documentazione ufficiale Node.js:** nodejs.org/api, il riferimento autorevole, sempre aggiornato.
- **MDN Web Docs:** developer.mozilla.org, per il JavaScript di base e le API web.
- **npm:** npmjs.com, dove esplorare le librerie disponibili e leggere i loro README.
- **Canali YouTube:** Fireship, Web Dev Simplified (in inglese), Codiez (in italiano): video brevi e ben fatti.
- **freeCodeCamp:** corsi gratuiti molto solidi per chi vuole rinforzare le basi.

Un consiglio prima di andare

Non sentirti in colpa se al primo giro non hai capito tutto. È normalissimo. La comprensione vera arriva con la pratica: scrivi codice, sbaglia, leggi gli errori, prova diverse soluzioni. Tutti i bravi sviluppatori che conosci sono passati esattamente attraverso lo stesso disorientamento iniziale.